AI Engineering and Claude Architect · Part IV, Production

AI Production Engineering, Lesson 1

Reliability: Retries, Rate Limits and Fallbacks

The monthly spend cap was hit at 2 a.m. Every retry layer treated it as a rate limit and kept trying, and the service stayed down until a human read the error body. Not every 429 is the same 429.

Module	AI10 of 13
Lesson	1 of 7
Prerequisites	HLD M11 (retries, breakers, backpressure), AI03 L1, AI08 L1
Exam relevance	Supports TS 5.3. API limits and fallback chains aren't tested directly
Reading time	~35 minutes

Contents

1. Reliability for LLM Calls

Lesson 1 of 7 · AI Production Engineering

Reliability for LLM Calls

Key idea

HLD M11's rules apply, with three changes: limits are counted in tokens as well as requests, a slow answer isn't a failed one, and a fallback model gives a different answer. Classify each failure before reacting. Retry in one layer only. Fail over before the first byte, since afterwards the user has already seen output.

> *Before: HLD M11 for backoff, jitter, breakers and bulkheads, which this lesson won't reteach. Teaches: classifying API errors, cache-aware limits, what retries really do in an outage, streaming failures, and fallback chains. Used by: L2, L6, L7.*

The hook: the 429 that retrying can't fix

The API has three different things that can look alike. From the rate limits and errors pages:

What happened	Status	Header	Does waiting a few seconds help?
Rate limit exceeded	429 `rate_limit_error`	Usually `retry-after`	Yes
Your tier's monthly spend cap	429 `rate_limit_error`, with `error_code` `enforced_spend_limit_reached`	No `retry-after`	**No.** Paused until the first of next month, or a tier increase
A spend limit you set yourself	**400** `invalid_request_error`	None	No

The second row is the trap, and the docs say so plainly: retrying, including the SDK's automatic retries, fails until access resumes.

So I tested this course's own code. `SpendCapTest` serves that exact error body from a local fake to the real SDK and `ClaudeModel` from AI08. Real output:

```
spend cap: requests=3 kind=RATE_LIMITED retryable=true sentToCaller=429
```

Three requests for an error that can't succeed, and AI08's mapping marks it retryable and passes a 429 to the caller. **A caller told "rate limited, try again" will keep trying for up to a month.** The fix is small: read `error.details.error_code`, and page a person. I've documented this with a test, and Exercise 1 fixes it.

The mirror-image trap sits in the third row. A limit you set returns 400, which most code files under "bad request, a bug in our code." It's really "we've run out of budget, as we chose to."

Intuition: three reasons a shop is closed

A shop's door is locked. It might be lunch, and the sign says back in 20 minutes. Or there's a queue out of the door. Or the rent hasn't been paid and the bailiffs have changed the locks.

A locked door looks the same in all three cases, and the right response is completely different: wait, come back later, or call someone. Reading the sign comes first.

Mechanism

What's different from an ordinary service

Property	Ordinary service	LLM API
Normal latency	Milliseconds	Seconds to minutes
Limit unit	Requests	Requests, input tokens, output tokens, by model class
Cost of a retry	Small	The whole prompt again, perhaps with a cache miss
Failure after success starts	Rare	Stream can break halfway (AI08 L2)
Fallback	Same answer from a replica	**Different model, different answer**
Deterministic?	Usually	No, so idempotency needs its own care

Classify first

Here's the policy in code, from `ReliabilityLab.classify`. Real output:

```
What to do with each error:
  429 retry-after    WAIT: for retry-after, then retry
  429 no header      WAIT: with backoff, and queue or shed new work
  429 spend cap      STOP: spend cap, retrying fails until access resumes. Alert a person
  400                FIX: the request, or a spend limit you set. Never retry unchanged
  401                FIX: credentials. Alert, and don't retry
  404                FIX: model ID or size. Don't retry
  529                RETRY: with backoff and jitter, then fall back
  504                RETRY: once, or stream the long request
```

Some facts behind this table, from the errors page:

→ **SDK retries.** The official SDK retries connection errors, rate limits and 5xx errors with exponential backoff, twice by default, honouring `retry-after` when present. AI08 L1 confirmed this behaviour with a fake.

→ **529 versus 429.** 529 `overloaded_error` means high traffic across all users. It isn't about your quota, so reducing your rate barely helps, and a different model may.

→ **Acceleration limits.** A sharp increase in your usage can also produce 429s. The docs advise ramping traffic gradually, which matters when launching a feature.

→ **Keep the request ID.** Every response has a `request-id` header. Log it, as support asks for it.

→ **Long requests.** Prefer streaming or Batch, especially for anything beyond 10 minutes. The SDK refuses non-streaming calls that it expects to exceed that.

Token buckets, and the cache bonus

The rate limits page says the API uses a token bucket, so capacity refills continuously. Limits may be enforced over shorter windows than a minute: its example is 60 requests a minute enforced as one a second. A burst can fail even when your per-minute average looks fine.

```
A bucket lets a burst through, then holds the average:
  30 requests at t=0: admitted 10
  one a second for 20 more seconds: admitted 20
```

Output and input are limited separately. OTPM counts real tokens as they're produced, and `max_tokens` doesn't reduce your quota, so a generous `max_tokens` carries no rate limit penalty.

Cached tokens don't count toward ITPM on most models. Only `input_tokens` and `cache_creation_input_tokens` count. So caching raises real throughput, which is a reliability benefit that's easy to miss. The docs give the 80% example, and this reproduces it:

```
Cache-aware input limit, with a 2,000,000 ITPM limit:
    0% cache reads ->    2,000,000 total input tokens a minute
   50% cache reads ->    4,000,000 total input tokens a minute
   80% cache reads ->   10,000,000 total input tokens a minute
   95% cache reads ->   40,000,000 total input tokens a minute
```

A cache miss therefore has two costs. It's billed higher, and it spends quota. A deployment that invalidates the cache can trigger 429s, even at the same traffic.

Read the headers. Responses carry `anthropic-ratelimit-*` values for limit, remaining and reset. A client that reads "remaining" can slow down before it's rejected.

Workspaces. You can give each workspace lower limits, which protects other workloads from a runaway one. The org-wide limit still applies. That's a coarse form of the tenant fairness in L6.

What retries do in an outage

I built a simulation: 80 requests a second, capacity of 100 a second, and a 20-second outage in the middle of two minutes. **It's a model with invented numbers, and it says nothing about any real API.** The numbers repeat exactly, from a fixed seed.

```
A 20-second outage, 80 requests a second, capacity 100 a second:
policy                              served  failed     sent  amplify  p95 wait
no retries                            8000    1600     9600    1.00x      0.0s
5 attempts, fixed 100 ms              8032    1568    15952    1.66x      0.0s
5 attempts, backoff                   9010     590    17572    1.83x     30.0s
5 attempts, backoff and jitter        8573    1027    16803    1.75x      9.6s
same, plus a breaker                  7926    1674     8006    0.83x      0.0s
```

Five readings, and the last two surprised me:

1. **Fixed 100 ms is the worst deal.** It sent 66% more calls, and recovered only 32 requests, because all five attempts land inside the outage.
2. **Backoff spreads attempts across time**, so some land after recovery. It recovered about 1,000 requests.
3. **Recovered work comes with a wait.** The 95th percentile wait went from nothing to 30 seconds. For a chat user, that's a failure with extra steps. For a batch job, it's a success.
4. **Jitter served fewer here than plain backoff.** With random delays, some attempts come sooner and fall inside the outage. Jitter's benefit is preventing a synchronized wave on recovery, and in this simulation capacity was never the constraint on recovery. So don't quote this as "jitter hurts." It shows that jitter solves a different problem from this one.
5. **The breaker served the fewest, and sent the fewest.** It's not there to recover more of these requests. It stops you hammering a failed dependency, returns errors at once, and frees you to do something better, such as a fallback or a queue. With nothing behind it, as here, it only fails faster.

Retries multiply across layers

```
Retries multiply across layers (attempts at each layer):
  [3] -> up to 3 calls per user request
  [3, 3] -> up to 9 calls per user request
  [3, 3, 3] -> up to 27 calls per user request
  [3, 1, 1] -> up to 3 calls per user request
```

The default stack is the SDK (three attempts), your service's retry library, and a gateway. **Choose one layer to own retries, and set the others to one attempt.** Otherwise an overload you're trying to survive gets 27 times worse, at your expense. If you retry in a gateway (L7), set the SDK's `maxRetries` to zero behind it.

Also pass a limit on total time. Three attempts at 60 seconds each is a three-minute wait, and the user left long ago.

Timeouts: first byte, not whole response

A long answer is normal, so one total timeout is a poor fit. Use several:

Timeout	Why	How to set it
Connect	Catches network failure fast	Seconds
Time to first byte	Catches a hung request, though a long think can delay it	Measure, by model and effort
Idle between events	Catches a stalled stream	Longer than the gap between `ping` events
Total	Protects the caller	By product need

The Claude Code gateway guide makes the same point. It aborts a stream when no bytes arrive for too long, and upstream `ping` events can be the only traffic during a long thinking pause. Don't filter them out.

Failing mid-stream

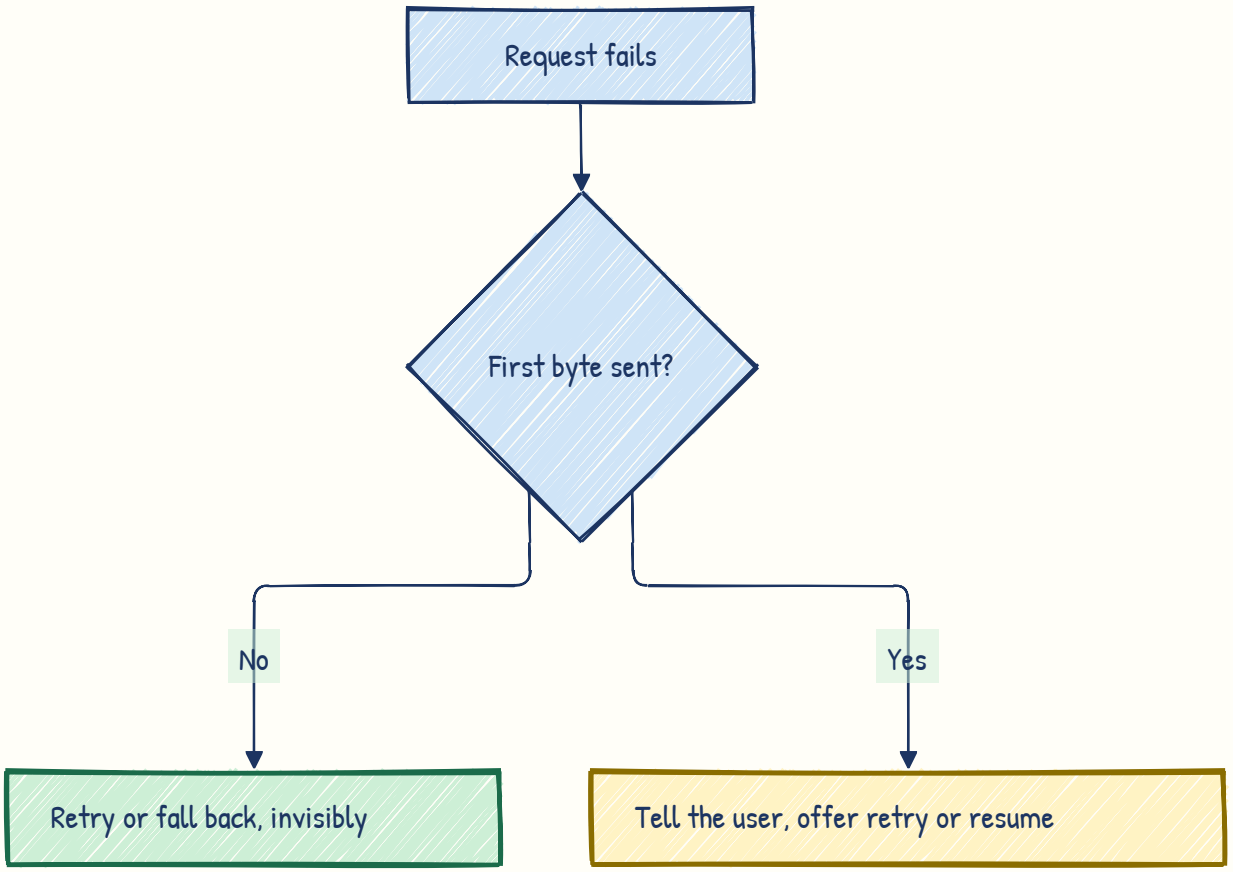


Figure 1. Before the first byte, retry freely. Afterwards, the user has seen text, so you need a decision and not a reflex.

Also note that an error can arrive inside a 200 response, as an SSE `error` event. The errors page says standard handling doesn't apply to it. Code that only checks the status code reports success. AI08 L2 covers this path.

Fallback chains

A fallback is a decision about quality as much as availability, so write down what each step does.

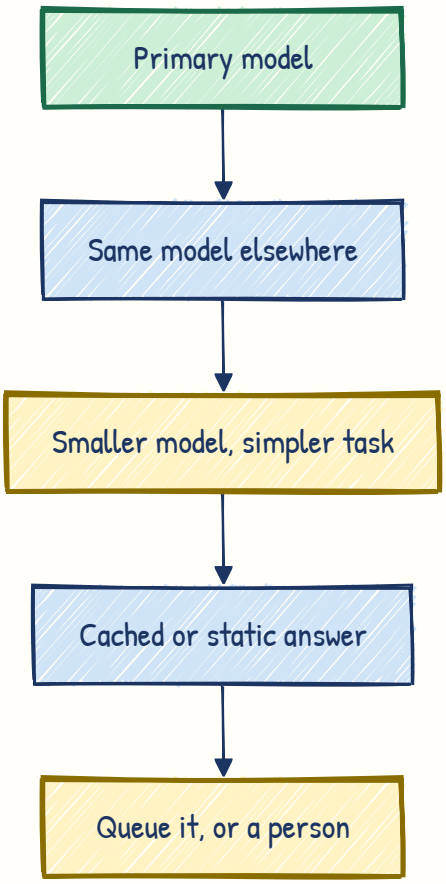


Figure 2. Each step costs quality or freshness, so the chain is a product decision. The last step is honest about being degraded.

Step	Quality	What to check first
Same model, another platform	Same, in principle	Different limits, features and retirement dates. **Run your evals there**
Smaller model	Lower	Does it pass your evals? Does it support your tools and parameters?
Cached or static	Stale	Is staleness acceptable here?
Queue	Same, later	Can this user wait?
A person	Often better, and slow	Is anyone there?
Say so honestly	None	"This feature is unavailable" beats a wrong answer

Features differ by model. The errors page has a long list of 400s that depend on the model: forced `tool_choice` fails on Opus 5.5 and Sonnet 5.5, thinking settings differ, and so on. A fallback that was never tested can fail with a 400 at the worst moment. **An untested fallback is a guess, so run it in CI and send it a little real traffic.**

Thinking blocks. Thinking blocks are bound to their conversation, and the errors page describes how a block from a model the target can't read is dropped rather than rejected. Don't assume a mid-conversation switch is seamless, and test yours.

Refusal fallbacks are a separate idea. The API has a server-side `fallbacks` parameter in beta, which re-runs a request on another model when safety classifiers decline it. That's about a policy decline, not an outage. The bundled API reference gives two different beta header dates for it, so read the current page before using it.

Shedding and queueing

When demand exceeds your limit, somebody has to wait or be refused. Choosing who is the design:

Traffic	During overload
Paid interactive	Serve
Free interactive	Queue briefly, then refuse with `retry-after`
Background and evals	Stop first, or move to Batch
Retries of old requests	Drop if the user has gone

Batch has its own limits, shared across models, and costs half as much. For anything that can wait, it's also a pressure valve.

Why the obvious fixes fall short

Obvious idea	Why it falls short
Retry all 429s	The spend cap never clears
Treat all 400s as bugs	Your own spend limit is a 400
Retry in each layer	Up to 27 calls
One long timeout	Can't tell slow from stuck
Fall back to any model	It may reject your parameters, or answer worse
Add more retries	More wait, and more cost, for the same outage
Raise `max_tokens` carefully for limits	OTPM counts tokens produced, so it isn't the concern

How Anthropic solved it

Mechanism	What's published	Source
SDK retries	Twice by default, exponential backoff, honours `retry-after`	Errors page
Token bucket	Continuous refill	Rate limits page
Cache-aware ITPM	Reads excluded on most models	Rate limits page
Headers	Limit, remaining, reset	Rate limits page
Workspace limits	Per-workspace rate and spend caps	Rate limits page
Service tiers	**Now legacy.** Priority Tier commitments are no longer sold. Contact sales for guaranteed capacity	Service tiers page
Server-side refusal fallback	Beta parameter	API reference

The Priority Tier row is worth noting if you've read older material. It targets 99.5% uptime for existing commitments, and isn't supported on the 5.x models. Don't design around it.

These describe the API's behaviour. Anthropic doesn't publish how its capacity system works internally, and I won't guess.

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: the SDK's default retries, calling the API directly.

Improved: classification, a single retry layer, and a tested fallback. New problem: each service builds this separately, and each holds its own view of remaining quota.

Production: a shared client library, or a gateway (L2), with priority queues.

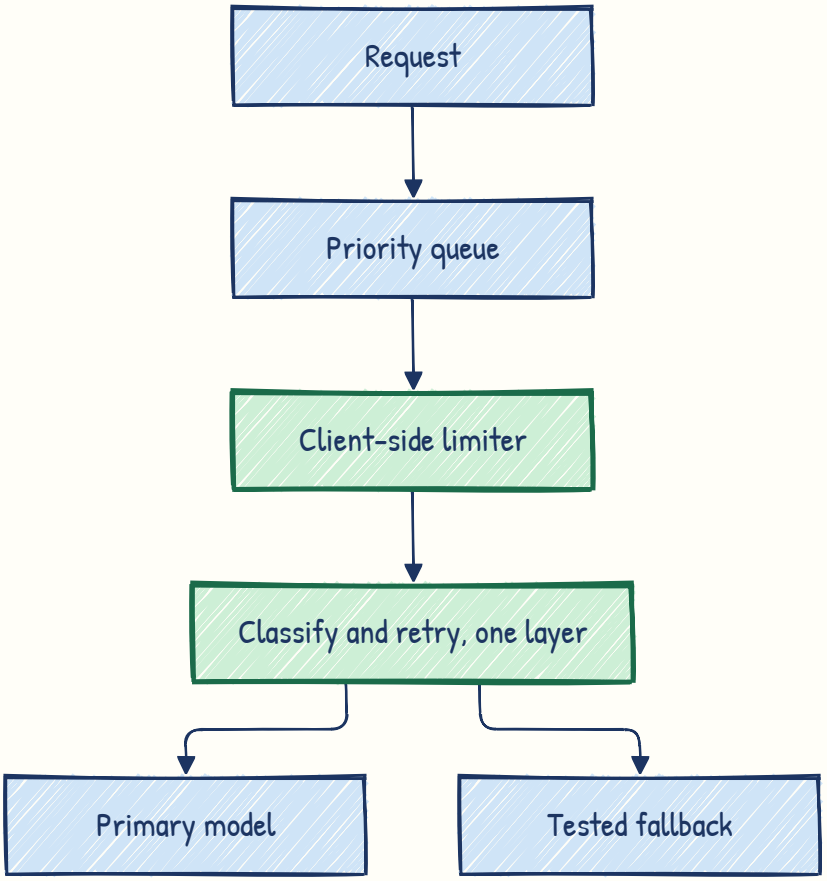


Figure 3. Overload hits the limiter first, not the provider. Errors are classified once, in one place.

Component	Why it exists	How it communicates
Priority queue	Decides who waits	In-process or a broker
Client-side limiter	Avoids being rejected, using the rate limit headers	Shared store, if several instances
Error classifier	One correct reading of each failure	Library or gateway
Single retry layer	Avoids multiplied retries	Config: other layers set to one attempt
Breaker per model	Stops calling a failed target, and moves to fallback	HLD M11
Fallback policy	Records what's allowed per task	Config, tested in CI
Alerts	Spend cap and auth need a person, now	Pager

Failures:

Failure	How you'd notice	What to do
Spend cap	`enforced_spend_limit_reached`, no `retry-after`	Page. Request a tier increase. Set your own limit lower, with an alert, so that you hear first
Key revoked or expired	401s	Page. Rotate
Model retired	404s (L6)	Migrate before the date. Alert on deprecated use
Sustained 529	Rate by model	Fallback, shed, and move work to Batch
Cache miss storm	Hit rate falls, 429s rise	Find the prefix change (L4), and roll back
Retry storm	Sent far above offered	One retry layer, plus a retry budget
Fallback rejects a request	400s after failover	Test in CI, and keep per-model parameter logic

Java

The classifier's ordering is the decision:

```
if (status == 429 && "enforced_spend_limit_reached".equals(errorCode)) {
    return "STOP: spend cap, retrying fails until access resumes. Alert a person";
}
```

The check comes before the generic 429 rule. A `switch` on status alone can't express this, which is why AI08's mapping missed it.

```
tokens = Math.min(capacity, tokens + (now - last) * perSecond);
```

That's the entire token bucket: continuous refill up to a cap, rather than a reset each minute.

```
double delay = policy.backoff() ? Math.pow(2, attempt.number()) : 0.1;
```

The simulation's only difference between "fixed" and "backoff" is this line, and it recovered about 1,000 more requests.

Trade-offs

Decision	Benefit	Cost
More attempts	Survives short failures	Wait, cost, and load
Long backoff	Spans the outage	Useless for interactive users
Breaker	Fails fast	Fails requests that might have worked
Fallback model	Stays up	Different quality, and needs evals
Second platform	Independent limits	Two sets of configs and features
Client-side limiter	Fewer 429s	Shared state, and can under-use quota

When not to bother: an internal tool with few users and a person watching can use the SDK defaults and show errors plainly. Don't build a fallback chain you won't test.

Common mistakes

Reading only the status code. The spend cap and the rate limit share one. Log and branch on `error.type` and `error.details.error_code`.

Common mistakes

Falling back to a model you've never evaluated. You may have swapped an outage for confident, worse answers, which is harder to notice.

Common mistakes

Keeping retries going after the user has left. Propagate cancellation. Otherwise you pay for answers nobody will read.

What to say in an interview

"I classify before reacting, since a rate limit, a spend cap and overload need different responses, and the spend cap can't be retried. One layer owns retries, because they multiply. I use separate timeouts for first byte and idle, and fail over only before the first byte. Every fallback is tested in CI, as a fallback model is a different product. And caching raises throughput, because reads don't count toward input limits on most models."

Certification lens

How this lesson maps to CCAR-F

TS 5.3 covers error propagation: recover locally from transient errors, and pass the type, the query, partial results and alternatives upward. **API limits, service tiers and fallback chains aren't in the guide.** Know them for real work, and don't expect exam questions on the header names.

These are original practice questions, not real or leaked.

Q Q1 · Exam style, scenario: Multi-Agent Research · Medium. A search subagent's backend times out after its retries. How should it report to the coordinator?

A. Return an empty result, so that the workflow continues

B. Terminate the whole workflow, to avoid partial reports

C. Return the error type, the query attempted, any partial results, and suggested alternatives

D. Return "search unavailable," and let the coordinator guess the rest

A **C** (TS 5.3). The coordinator needs enough information to decide. A hides the failure as "nothing found," B throws away everything else, and D omits what it needs to recover.

Q Q2 · Your service, your retry library and your gateway each make 3 attempts. The provider has a 5-minute overload. What happens, and what do you change?

A Each user request can become 27 calls, which adds load during the overload and raises cost. Pick one layer to retry, set the others to a single attempt, cap total time, and add a retry budget. Moving background work to Batch also releases capacity.

Q Q3 · Why can a release that changes one line of a system prompt cause 429s?

A It can invalidate the cached prefix. Those tokens then count toward input limits (AI03 L5), instead of being cached reads that don't count on most models. Traffic hasn't grown, and uncached input tokens per minute have. Watch the hit rate during a rollout.

Revision - Lesson 1

→ Three look-alikes: rate limit (429, wait), tier spend cap (429, no retry-after, stop), your own limit (400).

→ Classify first. Branch on error type and code, not just status.

→ SDK retries twice by default. One layer should own retries, or they multiply up to 27 times.

→ Limits are RPM, ITPM and OTPM, on a token bucket, possibly enforced over short windows.

→ Cache reads don't count toward ITPM on most models, so a cache miss spends quota too.

→ Simulation: fixed-delay retries waste calls. Backoff recovers work, with a wait. A breaker fails fast.

→ Use separate timeouts for connect, first byte, idle and total.

→ Fail over only before the first byte. Errors can arrive inside a 200 stream.

→ A fallback is a different product. Test it in CI.

→ Service tiers are legacy. Don't design around Priority Tier.

Your notes